ADR-005: Exponer el módulo MVP como una API REST

Decisión de arquitectura de software

Estado: Implementada en el código para inventario y préstamos. Los archivos que no son del MVP siguen llamándose directamente.

1. Problema que se busca resolver

La entrega final pide que el módulo MVP tenga endpoints REST. Después de la semana 13 el backend ya estaba en capas, pero la página seguía llamando a un archivo PHP distinto para cada acción: objetos.php, objetos_disponibles.php, prestamos.php, registrar_prestamo.php y devolver.php.

Con esa forma de trabajar no había una manera común de nombrar los recursos ni de usar los métodos HTTP. Cualquier método llegaba al mismo archivo, cada archivo manejaba la entrada por su cuenta y no existía una ruta para cambiar el estado de un equipo a mantenimiento o baja. Además, sin una API clara, otra pantalla u otro punto de préstamo no podía usar el mismo backend, y ese es el problema de escalabilidad (P3) de la semana 3.

2. Solución propuesta

Se crea un único punto de entrada para la API. El archivo .htaccess de la raíz envía todo lo que empiece por /api/ a src/api.php. Ahí están las rutas, y un Router escoge el controlador según el método HTTP y la ruta (patrón de controlador frontal).

Los recursos son dos: objetos (los equipos del inventario) y préstamos. Cada uno tiene su controlador, que no hace SQL: lee la petición y llama al servicio que ya existía.

Método y ruta	**Qué hace**
GET /api/objetos	Lista el inventario. Con ?estado=disponible devuelve solo los equipos que se pueden prestar.
PATCH /api/objetos/{id}/estado	Cambia el estado a mantenimiento, baja o disponible.
GET /api/prestamos	Lista los préstamos.
POST /api/prestamos	Registra un préstamo y responde 201.
POST /api/prestamos/{id}/devolucion	Registra la devolución.

Todas las respuestas son JSON. Los errores usan los mismos códigos en toda la API: 404 si algo no existe, 405 si la ruta no acepta el método, 409 si la operación choca con el estado actual y 422 si faltan datos.

El cambio manual de estado corre dentro de una transacción y no permite poner ni quitar el estado “prestado”. Ese estado solo cambia al registrar o devolver un préstamo, para respetar lo definido en el ADR-002 y el ADR-004.

3. Cambios realizados en el código

Antes	Después
objetos.php y objetos_disponibles.php	ObjetoControlador (src/Inventario/Http/), con GET /api/objetos y el filtro ?estado=disponible.
prestamos.php y registrar_prestamo.php	PrestamoControlador (src/Prestamos/Http/), con GET y POST /api/prestamos.
devolver.php recibía el número del préstamo en el cuerpo.	POST /api/prestamos/{id}/devolucion: el préstamo va en la ruta.
No había forma de pasar un equipo a mantenimiento o baja desde la página.	PATCH /api/objetos/{id}/estado, que usa InventarioServicio::cambiarEstado(). En Inventario hay un selector para usarlo.
Cada archivo era una entrada distinta al backend.	src/api.php y src/Compartido/Http/Router.php reciben todas las rutas /api/.
index.html llamaba a src/*.php para inventario y préstamos.	index.html llama a api/... Los archivos que no son del MVP se siguen llamando igual.

4. Atributo de calidad que mejora

Mantenibilidad. Todas las rutas del MVP están declaradas en un solo archivo y los errores se manejan igual en todas. Agregar un endpoint nuevo es una línea en api.php y un método en el controlador, sin tocar los servicios ni los repositorios.

También deja una base para la **escalabilidad** (P3): cualquier otra pantalla, aplicación o punto de préstamo puede usar la misma API. La API está documentada en el README y en Swagger (docs/api/).

5. Otras alternativas que se descartaron

- Usar un framework como Slim o Laravel para las rutas. Se descartó por la misma razón del ADR-001: implicaba aprender una herramienta nueva, y un Router de unas pocas líneas era suficiente para cinco rutas.

- Dejar los archivos sueltos y solo cambiarles el nombre. No resolvía el uso de los métodos HTTP ni el manejo común de errores.
- Usar una sola dirección con un parámetro, como api.php?accion=prestar. Funciona, pero no sigue el estilo REST de recursos y métodos que pide la entrega.

6. Ventajas y costos

- **Ventaja:** la API queda documentada en el README y en Swagger, y para hacerla solo cambió la capa de controladores. Los servicios, el dominio y los repositorios quedaron igual.
- **Costo:** las direcciones /api/ dependen de que Apache lea el archivo .htaccess (mod_rewrite). En XAMPP ya viene activo; si no lo estuviera, la API se puede usar con la dirección larga src/api.php/objetos. Además, mientras no se migren los otros diez archivos, conviven dos formas de llamar al backend.

7. Cómo se verifica la solución

- Abrir http://localhost/prestamos/api/objetos en el navegador. Debe mostrar el inventario en JSON.
- Desde la página, registrar un préstamo, cambiar el estado de un equipo y hacer una devolución. Todo debe funcionar igual que antes, ahora usando la API.
- Abrir http://localhost/prestamos/docs/api/ y probar los endpoints desde Swagger.
- Pedir una ruta que no existe (por ejemplo GET /api/usuarios) y usar un método que no se acepta (DELETE /api/prestamos). Deben responder 404 y 405.